Sistemi Operativi

Leonardo Mengozzi

Titoletti indice link a rispettive sezioni, in alto a sinistra "←Indice" link a pagina Indice.

Contents

1 Bash **2**
Comandi Variabili . . . 2
Comandi File speciali . . . 2
Comandi Directory . . . 2
Comandi Tilde Expansion . . . 2
Comandi Privilegi . . . 2
Comandi Controllo Comandi . . . 3
Comandi EXP aritmetica . . . 3
Comandi Conditional EXP . . . 3
Comandi Scripting . . . 3
Comandi Subshell . . . 4
Comandi File Descriptor . . . 5
Comandi Uso File Descriptor . . . 5
Comandi stdin e stdout . . . 5
Comandi Utilities . . . 5
Comandi Costrutti controllo flusso . . . 6
Comandi Process Identifier . . . 6
Comandi Espansione Parametri . . . 8
Comandi Gestione risorse . . . 8

2 Crittografia **8**
2.1 Crittografia Simmetrica (chiave segreta) . . . 8
2.2 Crittograffia Asimmetrica (chiave pubblica) . . . 9
2.2.1 Funzioni Hash Crittografiche . . . 9
2.3 MAC e Firma digitale . . . 9
2.4 GPG e PGP . . . 10
Comandi GPG . . . 10

3 Programmazione Concorrente **11**
Comandi Multi-processo . . . 11
Comandi PIPE . . . 12
Comandi Multi-thread . . . 12
3.1 SJF . . . 13
3.2 Produttore e consumatore . . . 13
3.3 Buffer limitato . . . 13
3.4 Filosofi orientali a cena . . . 14
3.5 Message Passing . . . 15
3.6 Concorrenza in C . . . 15
Comandi Funzioni semafori . . . 15

4 Esercizi Scheduling **16**

5 File Descriptors **16**

6 Risorse **17**
6.1 Grafo di Holt Generale (classi e processi) . . . 17
6.2 Detaction and recoverty - Caaso 1 . . . 17
6.3 Detaction and recoverty - Caaso 2 . . . 18
6.4 Detaction and recoverty - Knot . . . 18
6.5 Stato SAFE - algoritmo del banchiere . . . 18

7 Gestione Pacchetti in Linux **18**
7.1 Concetti chiave 18
7.2 Installazione Package 18
7.2.1 Posizione Repository 19

1 Bash

Variabili	Descrizione
PATH	V. Ambiente: Modificabile, sequenza di percorsi assoluti, divisi da ":", di directory contenenti eseguibili (lanciabili senza digitare path). Ricerca secondo ordine specificato in PATH, si ferma a primo eseguibile con nome uguale. Eventuale ErrorNotFoutd. Altre variabili d'ambiente: $HOME, $USER, $SHELL, $TERM.
env	Visualizza l'elenco delle variabili d'ambiente.
IFS=$' \t \n'	Contiene caratteri separatori della parole negli elenchi.
/dev/null	File speciale che scarta tutto quello che gli viene scritto.
LC_CTYPE	Imposta il tipo locale dei caratteri. Usata da `cut` e `ws` per capire numero byte caratteri.
strace *comando*	Dice la sistem-call usata.
$RANDOM	Restituisce un numero intero casuale tra 0 e 32767.

File speciali	Descrizione
/etc/passwd	Righe sono info ogni utente divise da ":".
/etc/shadow	Righe sono password utente codificate.
/etc/group	Righe sono info ongi gruppo divise da ":".
/usr/bin/passwd	Cambia la pass utente.

Directory	Descrizione
cd *percorso*	Sposta logicamente la directory corrente in quella specificata, secondo un path asssoluto o relativo.
mkdir *nomeDir*	Crea una nuova directory.
touch *nomeFile.estensione*	Crea un file vuoto nella dyrectory corrente.
rmdir *nomeDir*	Rimuove una directory, solo se è vuota.
rm *file*|*dir*	Rimuove una directory vuota o un file.
rm *-r file*|*dir*	Elimina ricorsivamente sotto cartelle e file.
rm *-f file*|*dir*	Non fa chiedere le autorizzazioni di eliminazione.
mv file1 file2|dir	Rinomina file1 in file2 o sposta file1 nella directory specificata.
cp file1 dir	Copia file1 nella directory specificata.
ls *nomeDirOpzionale*	Visualizza i files/direcotry contenuti nella directory corrente o di quella specificata.
ls -a *nomeDirOpzionale*	Mosta anche file nascosti (anche ., ..).
ls -l *nomeDirOpzionale*	Mostra informazioni file e relativi permessi nel formato: 1°carattere tipo file (-file, d dyrectory, c collegamento seriale, b device a blocchi), 9 caratteri successivi terzine di permessi (r read,w write,x/s execute) per proprietario, gruppo, altri utenti.
ls -h *nomeDirOpzionale*	Rende i dati più leggibili.
ls -d *nomeDirOpzionale*	Fa applicare il comando alla directory stessa/specificata e non ai file.
ls -R *nomeDirOpzionale*	Mostra ricorsivamente contenuto sotto directory.
pwd	Visualizza il percorso assoluto, da / fino alla directory corrente.
which *comando*	Cerca in PATH il comando e se lo trova mostra il path in cui si trova.

Directory `/proc` : info sui processi, con una dir. per ogni processo attivo, create e cancellate continuamente. Directory sulla RAM `/proc/` : contiene i processi in esecuzione e nella sottodirectory `fd` i file descriptor aperti.

Tilde Expansion	Descrizione
~/...	Tilde espansa con il percorso assoluto della home directory dell'effective user. Valido caso con solo ~ e solo ~/.
~userName/...	Tilde e userName espansi con il percorso assoluto della home directory dell'utente specificato. Valido caso con solo ~**userName/**.

Privilegi	Descrizione
chmod *privilegi script.sh*	Modifica permessi file mediante formato numerico (terna 0-7). Ogni numero è la somma dei valori associati hai permessi di r(4), w(2), x/s(1). Ordine: proprietario, gruppo, altri utenti. Può diventare un quartetto aggiungendo per primo l'identificatore numero dei privilegi di *setuid, setgid, sticky bit.*
chgrp *nuovoGruppo file*	Modifica il gruppo di appartenenza di un file.
chown *newOwner nameFile*	Modifica il proprietario (e anche gruppo) di un file.
whoami	Dice all'utente corrente le sue informazioni.
sudo *comando*	fa eseguire il comando come administratore, può essere chiesta userPass. Solo utenti gruppo sudo (gestito dall'admin) possono usarlo.

Controllo Comandi	Descrizione
\	Disabila interpretazione per il carattere successivo (ex: andata a capo) permettendo di stamparlo.
"..."	Delimita un argomento e non fa interpretare nessun comando a eccezzione espansione di variabili e esecuzione di comandi.
'...'	Delimita un argomento e non fa interpretare nessun comando.
$'...'	Espande backslash-escaped direttamente nella stringa espansa in una single-qouted stringa. Backslash-escaped: \a, \e, \f, \r, \v, \', \b, \E, \n, \t, \\, \", \nnn, \xHH, \cn.
pre{s1,...}post	Stringa di testo racchiusa fra separatori (spazio, tab, a capo) con coppia di graffe (non precedute da $) e senza separatori. Le stringhe racchiuse dalle graffe vengono composte con il preambolo (pre) e postscritto (post), che sono opzionali. Sono annidabili (quelle più esterne eseguite per prime). Vengono eseguite prima le brace expansions delle variable expansions.
pre{a_1..a_2}post	Lettere da a_1 a a_2 nell'alfabeto.
pre{n_1..n_2}post	Numeri compresi tra n_1 e n_2.
*	Sostituito con una qualsiasi sequenza di caratteri anche vuota.
?	Sostituito con un singolo carattere (no spazio vuoto).
[c1c2...]	Sostituito con solo uno dei caratteri specificati in elenco. Annidabili.
[a_1..a_2]	Sostituisce con solo una lettere da a_1 a a_2 nell'alfabeto.
[n_1..n_2]	Sostituisce con solo un numero dei numeri compresi tra n_1 e n_2.
[:digit:]	Sostituisce con una cifra.
[:upper:]	Sostituisce con un carattere maiuscolo.
[:lower:]	Sostituisce con un carattere minuscolo.
history	Visualizza comandi, numerati, precedentemente eseguiti, anche da shell precedentemente chiuse. Con !NUMERO lancio il comando corrispondente nell'elenco di history. Con !STRINGA lancio il comando più recente che corrisponde alla stringa.

EXP aritmetica	Descrizione
((...))	Valuta una stringa come un espressione aritmetica (+,-,*,/,%, (), !, &&, ||) di soli interi. Racchiude un espressione più eventualmente un assegnamento. Si possono usare variabili nell'espressione ($variabile). Exit statuts 0=true, altro=false. Non contiene espressioni condizionali e comandi. Per le operazioni in virgola mobile usare **bc**.
$((...))	Come operatore ((...)) ma è concatenabile con stringhe tramite " ".

Conditional EXP	Descrizione
[[...]]	Restituisce exit status 0=true, altro=false. • Non può contenere conmandi, word splitting, brace expansion, pathname expansion. Non esegue assegnamenti a variabile, annidamenti di espressioni condizionali. • Può contenere variable expansion, solo $(()), command substitution (se non genera comandi), process substitution, quote removal. Questi solo per gli operandi. • Può andare a capo. • Si possono usare gli operatori logici !, &&, ||, (). Operatori unari/binari non quotabili ne generaibili da command substitution: • Operazioni sui file: -e (esistenza), -d (cartelle), -f (file), -h (link), -r (leggibile), -s (size>0), -t (fd open e riferisce un terminale), -w (scrivibile), -x (eseguibile), -O (possesso effettivo utente), -G (possesso effettivo gruppo), -L (=h). Confronto date ultima modifica file: f1 -nt f2 (f1 nuovo o f2 inesistente), f1 -ot f2 (f1 veccio o inesistente). • Operatori aritmetici: -eq (==), -ne (!=), -le (<=), -lt (<), -ge (>=), -gt (>). • Operatori lessicografici: ==, =, !=, <, <=, >=, >, -z (size==0), -n (size!=0). -o da vero se opzione è abilitata per la shell. Versioni vecchie: [..], test ... no a capo (semmai \), -a, -o, no (). Mettere sempre spazi prima e dopo quadre.
cmd1 ||cmd2	Esegue cmd1 e solo se cmd1 fallisce (exit status≠0) esegue cmd2.
cmd1 && cmd2	Esegue cmd1 e solo se cmd1 ha successo (exit status=0) esegue cmd2.

Scripting	Descrizione
clear	Pulisce la CLI. Non modifica variabili create.
./ *comado*	Esegue comando presente nella directory corrente (percorso relativo). Alternativa è inserire il percorso relativo o aggiungere il percorso assoluto alla variabile PATH.
nome=*valore*	Simboli con nome e valore, stringa modificabile, alfanumerici casesensitive. No spazi prima o dopo "=". Sono d'ambiente o ex-novo locali. Solo la bash in cui sono create le variabili le può usare. I programmi lanciati dalla bash hanno una speudocopia della bash.
unset *nomevariabile*	Elimina una variabile esistente (vuota o no). Quotare ("...") sempre variabili per evitare errori con variabili vuote o inesistenti.
${variabile}	Espanzione di variabile, ovvero la stostituisce con il suo contenuto. Graffe opzionali se nome variabile seguito da uno spazio.
${#variabile}	Restituisce il numero di caratteri del contenuto della variabile.
${!variabile}	Fa l'espanzione della variabile che contiene il nome d'un altra variabile con il valore di quest'ultima (riferimento indiretto). Dalla versione 2 di bash.
cmd1 ; ...	Separatore di più comandi, e dei rispettivi argomenti, scritti sulla stessa riga di comando e eseguiti dopo la terminazione del precedente (lista di comandi). L'exit status è quello dell'ultimo comando lanciato. Se racciusi da () eseguiti in una sub-schell.
#...	Commeto.
#!...	Scritto nella prima riga indica quale interprete deve eseguire lo script. Se non specificato viene usato quello corrente.
script.sh c1...	I parametri sono un'insieme ordinato di caratteri separati da spazi successivi al nome del programma. Sono immodificabili dopo la sostituzione dei metacaratteri (*, ?,ecc). Riga di comando = nomeProgramma + parametri. Nella riga di comando gli elementi sono indicizzati da 0 (nomeProgramma): • **$#** Contiene il numero di parametri passati. • **$n** Accede all'n-esimo parametro a partire da indice 0. • **$*** Tutti argomenti concatenati e divisi da spazi. • **$@** Vettore di argomenti quotati ("..."). I parametri $* e $@ sono identici se non quotati (concatenazione di argomenti separati da " "). Se quotati $* quota tutti gli argomenti assieme mentre $@ quota singolarmente ogni argomento. $@ è usato per passare parametri a comandi dentro a degli script.
̀comando|./script.sh`	**Command substitution**. Sostituisce (a run-time) nella riga in cui è specificato il comando o script con l'output (stdout). Comando alternativo $(./script.sh)
$?	Modificato alla terminazione di ogni script, contiene l'*exit status*.

Subshell	Descrizione
bash	Crea una shell figlia (interattiva non di login). Eredita dal padre: dir. corrente, copia variabili d'ambiente. Non sono ereditate le variabili locali. Creata in automatico per comandi raggruppati, script, processi in background. I comandi built-in sono eseguiti in shell corrente/padre.
bash *-c script.sh*	Crea shell non interattiva.Vi si può far eseguire uno script specificato.
bash *-l* |*–login script.sh*	Crea shell interattiva di login. Vi si può far eseguire uno script specificato.
export nomevar |nomevar=valore	Crea una variabile d'ambiente. Una shell figlia ne riceve una copia modificabile che non influenza la variabile d'ambiente del padre.
var=val ... comando	Scrivendo delle assegnazioni prima dell'esecuzione di un comando si creano delle var. d'ambiente solo per l'imminente subshell. Non saranno ereditate da successive subshell.
. |source *script.sh*	Esegue lo script nella shell corrente. Utile a impostare/modificare variabili shell. Ignorata prima riga opzionale e eseguito con interprete corrente.
exit	Termina bash corrente, elimina l'ambiente e sale alla padre.
exit *exitStatus*	Termina lo script restituendo un valore intero [0-255] per indicarne l'esito di terminazione. 0 indica esecuzione terminata senza errori, qualcosaltro indica un'errore. Viene restituito alla shell esecutrice.
top	Mostra in tempo reale processi in esecuzione e risorse di sistema usate.
ps	Mostra i processi in esecuzione.
ps -all	Mostra i processi in esecuzione con informazioni aggiuntive (PID, PPID,ecc).
set	Visualizza sia variabili locali che d'ambiente della shell corrente (anche funzioni di shell). I parametri [re]settano dei comportamenti della shell:
set +o *comando*	Disalibilita il comando (tipo history).
set -o *comando*	Abilita il comando (tipo history).
set -a	Successive variabili create/modificate diverrano d'ambiente e ereditate da shell figlie. Per [ri]definire variabili locali usare **export -n *variabile***.
set +a	Successive variabili create/modificate diverrano locali (non ereditabili da shell figlie), modalità di default.

File Descriptor	Descrizione
$$	Variabile con PID della shell corrente. Utile per esplorare /proc/ e fd vari processi.
exec {*n*}*modalità file.est*	Apre file e l'associa al fd scento dall'utente (n). Modalità: < (lettura), > (scrittura), >> (aggiungi in coda), <> (lettura e scrittura).
exec {*var*}*modalità file.est*	Apre file e l'associa al fd scento dall'os inserito in var.
exec {n}>&-	Chiusura di file con fd=n.
exec {*var*}>&-	Chiusura di file con fd contenuto in var.

Uso File Descriptor	Descrizione
program *modalità file|prog2 ...*	Ridirezionamento std di program. Modalità: < (strin da file), > (strout sovrascrive file), >> (srtout accoda a file). Possibili più ridirezionamenti contemporaneamente (ordine non conta). Il fine file da tastiera si fa con "Ctrl+D".
program *modalitàEfd* file|prog2 ...	Ridireziona il file o prog su uno specifico fd N di program. Modalità: N>, N>>, <N.
program 0/1/2/&> file|&0/1/2	Ridireziona lo std (&=stdout+stderr) indicato di program nel file indicato sovrascrivendo o in un altro str. &> ridireziona solo in fle.
program N>&M ...	Fa puntare lo stream di N allo stesso stream di M (N e M sono fd). Si possono fare più ridirezionamenti (fd e file, fd e fd) in simultanea. Sono eseguiti da sinistra a destra.
prog1|prog2	pipe: collega stdout di prog1 allo stdin di prog2 (usa speudo-file temporaneo). Unidirezionale sinDes.
prog1 |& prog2	Ridireziona lo stderr nello stdout di prog1 e lo passa allo strdin di prog2.
costrutto *modalità file*	Applica il ridirezionamento a tutti i comandi contenuti nel costrutto e alla condizione.
program << *word*	Word specificata è delimitatore di fine input da passare al comando. Nell'input la word deve essere a inizio di una riga con solo essa.
program <<< *word*	Ridireziona in input la word. Se si qouta ("...") si possono passare più dati.
(cmd1;...)	Comandi eseguiti in subshell con stdin/out/err condivisi e concatenati. L'exit status è quello dell'ultimo comando eseguito. Permette ridirigere più comandi simultaneamente.

stdin e stdout	Descrizione
read *var*	Legge dallo stdin o file una sequenze di caratteri (fino all'INVIO o a capo) e la inserisce nella variabile passata. Ingnora spazi e tabultazioni inizio e fine riga (ameno che si setti IFS=""). Exit status 0=lettura eseguita, >0=eof (var="") o ultima riga senza \n (var="ultimi caratteri"). Necessario controllo con #var. Una read parte dalla riga precedente, se non era stata finità di leggere (fino a \n).
read *-u ${var1} var*	Leggere dal file descriptor var1.
read *-n numero var*	Numero massimo di caratteri da leggere.(Considera spazi e tabulazioni).
read *-N numero var*	Numero esatto di caratteri da leggere.(Considera spazi e tabulazioni). Considera \n come semplice carattere. Usa prossime righe se attuale ha caratteri insufficente.
read *var1 ...*	Alle variabili viene assegnata i-esima parola della riga tranne per l'ultima variabile che riceve la stringa rimanente. Alle variabili in eccesso viene assegnato valore vuoto.
echo *testo*	Visualizza a video la sequenza di caratteri passata fino al primo "INVIO". Se quoto l'input si disabilità l'interpretazione dei caratteri speciali e andate a capo.
echo *-e testo*	Stampa i caratteri speciali (\...).
echo *-n testo*	Non fa andare a capo.

Utilities	Descrizione
head -n *n file*	Mostra le prime n righe del file. Default 10.
tail -n *n file*	Mostra le ultime n righe del file. Default 10.
sed *'s/old/new/[g|n];...' file*	Sostituisce il testo old (o più testi orinali con [o1, o2, ...]) con quello new, solo per la prima occorrenza. Può richiere l'opzione `-r`. • Con l'opzione g si applica per tutte le occorrenze in una riga oppure all'n-esima occorrenza. Si possono concatenare più sostituzioni d'applicare sulla stessa riga. • ^ significa inizio linea. Va all'inizio di new. • **$** significa fine riga. Va alla fine di new. • **.** significa un carattere qualsiasi. **.{n}** significa n caratteri qualsiasi. **.*** significa 0 o più caratteri qualsiasi. • **[...]** caratteri o insiemi di caratteri (a1-an) da sostituire. backreference: In "originale" si specificano tra () stringhe riusabili in "nuovo" traminte i-esima backreference.
cut *-c S-D* file	Estrae colonna/intervallo di caratteri, da ogni riga, dall'indice S all'indice D (inclusi). S o D si possono omettere e si prende dall'inizio o dalla fine fino a S o D. L'indice parte da 1. Posso concatenare più intervalli dividendoli con ",".
cut *-d '...' -fN* file	Divide le righe in parole secondo il carattere delimitatore indicato poi seleziona la parola con l'indice f-ennesimo di ogni riga.
cat *nomeFile.estensione*	Visualizza il contenuto del file.
grep *stringa* [*file*]	Filtra le righe contenenti la stringa passata (che sarà evidenziata), anche se contenuta in altre parole, dal testo che sarà successivamente scritto (terminato da "Ctrl+d") o dal file passato opzionalmente. Spesso usato come secondo comando di |per filtrare output di altri comandi.
grep -v *stringa* [*file*]	Filtra righe senza parola da evidenziare.
grep -i *stringa* [*file*]	Disabilità il caseSensitive.
grep -h *stringa* [*file*]	Disabilita la visualizzazione del nome del file dove è avvenuta la ricerca (nel caso si cerci in più file). (-H è equivalente).
find *path*	Trova percorsi di file in sotto alberi della memoria del path indicato.
find *path* -type d	Trova solo directory.
find *path* -type f	Trova solo file.
find *path* -maxdepth n	Limità la profondità di ricerca il numero di livelli indicato (0 si limita alla cartella indicata).
find *path* -mindepth n	Indica il livello minino di ricerca (1 è la cartella indicata).
find *path* -name "..."	Definisce un vincolo sul nome del file da trovare.
find *path* -iname	Disabilità il caseSensitive.
find *path* -exec ... '{}' ... \;	Istruzioni da fare quando viene trovato un elemento. L'output di find viene messo al posto di '{}'.
find *path* -print	Obbliga la stampa dei file trovati anche quando find non stamperebbe.
wc	Dice del testo passato il numero di righe (-l), parole (-w) e caratteri (-c).
tee file	Stampa a video e salva l'output di un comando (che gli manda l'input, tipo con —), sul file indicato.
sort *file*	Ordina alfabeticamente le righe del file.
sort -k n	Ordina le righe alfabeticamente secondo la colona di stringhe specificata.
uniq *file*	Rimuove le righe duplicate consecutive. (Bisogna sortare prima).
uniq -c *file*	Rimuove le righe duplicate consecutive e conta quante sono.

Costrutti controllo flusso	Descrizione
for varName in elencoParole ; do listCommand ; done	Dopo il for variabile è usabile.
for ((exp1 ; exp2 ; exp3)) ; do listCommand ; done	Le exp sono espressioni valutate aritmeticamente automaticamente.
if listA ; then listB ; [elif listC ; then listD ; ...] ... [else listZ ;] fi	
while list ; do list ; done	

In tutti i costrutti l'exit value è o quello della lista do comandi o 0 se nessun comando viene eseguito. Espressioni condizionate su file o variabile: [condizione di un file]. Valutazione di un espressione matematica applicata a variabili d'ambiente: ((istruzini con espressione)).

Process Identifier	Descrizione
$$	Variabile espansa col PID (identificatore numerico univoco di processo) della shell corrente. Col raggruppamento di comandi `(...; ...)` viene espanso col PID della shell più esterna.
$BASHPID	Da in ogni caso il PID della shell corrente. Comando poco portabile.
script &	Lancia processo in background. Si omette ; (& già terminatore comando). `$!` ha il pid del processo lanciato in bg (anche se sganciato). Non è aggiornato se faccio bg di un processo sospeso da ˆZ.
jobs	Mostra lista procesi backgroud e sospesi con rispettivo indice di job. Con `%indiceJob` si gestisce il job.
ˆZ	Sospende processo in foreground. Lancia il segnale SIGTSTP
ˆC	Termina processo in foreground. Lancia il segnale SIGINT.
bg [%n]	Riprende l'esecuzione in background di un Job sospesto/del Job specificato. Lancia il segnale SIGCONT.
fg [%n]	Riprende l'esecuzione in foreground di un Job sospesto/del Job specificato. Lancia il segnale SIGCONT.
kill [-valore] pid|%n [-1]	Usato per termina il processso di PID o JobId indicato. In generale manda un segnale (interruzione software per notificare evento asincrono), default SIGTERM=15. In generale un segnale è accettato solo se proviene dall'effective user e poi viene gestito/ignorato dal processo interessato (SIGKILL=9 non può essere ignorato). Con -1 si fanno terminare tutti i figli diretti della bash.
kill -l	Da tutti segnali lanciabili.
killall 'nomeProc'	Termina tutti processi con lo stesso nome o mandare a più processi uno stesso segnale. Manda SIGTERM di default. `-r` Permette di usare le espressioni regolari.
disown pid|%n, ...	Sgancia jobs dalla shell. Viene sganciato l'ultimo job messo in bg se non si specifica nulla.
disown -r	Sgancia tutti job in running.
disown -a	Sgancia tutti job in running e suspend.
nohup *script* &	Lancia in bg e sgancia ultimo processo messo in bg.
%%|%+|%	Riferisce job corrente (ultimo interrotto) in foreground o avviato in backgourd.
%-	Riferisce penultimo job interrotto in foreground o avviato in backgourd.
trap "azione" segnale1,...	Imposta "solamente" l'azione (funzione o lista comandi) da svolge alla ricezione di certi segnali predefiniti. Funzione: `nomeFunzione(){...}`
wait pid|%n, ...	Fa attendere la shell la terminazione del processo figlio di pid o JobId indicato (o un insieme di processi). Ha exit status l'exit status dell'unico o dell'ultimo processo terminato da attendere. `wait` senza parametri fa attendere tutti figli e da exit status 0.
sleep n	Fa attendere n secondi al programma.

Terminologia:

- **Processo**: Minima unità completa ed'autonoma d'elaborazione. Composto da più thread operanti in un contesto = spazio d'indirizzamento + tabella dei descrittori di file aperti.
- **Gruppo di processi**: Insieme variabile di processi avviati da un processo padre.
- **Terminale di controllo**: astrazione della console=video+tastiera (stdni, stdout, stderr) opzionale per un processo.
- **T.c. del gruppo di processi**: terminale controllo padre ereditato dai figli, ameno di sganciamenti volontari.
- **Sessione**: Creata e legata a un terminale di controllo da una shell (leader sessione). sessionId=PID leader sessione. Shell prende dal terminale di controllo stdin, stdout, stderr.
  Processi figli apparteranno alla sessione, ameno di distacchi, e condividono terminale di controllo.
  Un terminale è t.c. per una sola sessione alla volta. La ciusura del t.c. comporta la terminazione dei processi del gruppo (SIGHUP).
- **Processo foregroud**: Controlla terminale da cui è stato lanciato (stdin collegato e impedisce esecuzione fino alla sua fine ad'altri programmi).
  Uno processo foregroud alla volta.
- **Processo backgroud**: Eseguito in parallelo rispetto bash madre (bash può leggere e eseguire indipendentemente). Hanno una copia fd della bash.
  Terminati, ameno di sganciamento, alla chiusura del terminale della bash madre (stdin condiviso).
  Più processi backgroud alla volta.
- **Job control**: Componente che cambia stato processi da background a foreground e viceversa.
- **Jobs**: Processi in background o figli sospesi di una shell
- **Processo zombie**: Processo figlio terminato (risorse state rilasciate dall'os) di cui l'os mantiene il pcb (pid, exit status, ecc).
  Del tutto eliminato quando padre fa la wait (rilascio pcb).
- **Processo orfano**: Processo figlio di cui il padre è terminato senza fare wait. Viene adottato dal processo

`init` (PID=1) di cui farà la wait periodicamente (rilascio pcb).

Espansione Parametri	Descrizione
${*variabile*%%pattern}	Rimuove il più lungo suffisso.
${*variabile*%pattern}	Rimuove il più corto suffisso.
${*variabile*##pattern}	Rimuove il più lungo prefisso.
${*variabile*#pattern}	Rimuove il più corto prefisso.
${#*variabile*}	Restituisce il numero di byte della variabile.
${*variabile*/pattern/string}	Sostituisce la prima occorrenza da sinistra del pattern con la stringa indicata. Caratteri speciali da inserire dopo primo "/": • /: Fa sostituire tutte sotto strighe e non solo la prima. • #: Fa sostituzione solo se pattern è all'inizio. • %: Fa sostituzione solo se pattern è alla fine.
${*variabile*:ofset[:length]}	Restituisce la sotto stringa dal carattere i-esimo (ofset da 0). Opzionalmente si indica la lunghezza della sotto stringa da estrarre (length). Sia ofset che length sono valutati aritmeticamente.
${!*variabile**}	Da le variabili il cui nome inizia col prefisso indicato.

Danno una parte o una parte modificata (sostituiscono una parte in genere) del contenuto di una variabile. Il contenuto della variabile non viene modificato. Def. **pattern**: Stringa da matchare nel contenuto della variabile. Può contenere wildcard e variabili.

Gestione risorse	Descrizione
sudo apt-get update	Aggiorna la lista locale dei pakage.
sudo apt-get upgrade	Aggiorna i package.
sudo apt-get install *nomepkg*	Installa il package se è nella lista locale.
sudo apt-get install –reinstall *nomepkg*	Reinstalla il package sovrascrivendo l'attuale installazione.
sudo apt-get install -y *nomepkg*	Installa anche dipendenze necessarie.
sudo apt-get purge *nomepkg*	Disinstalla il pakage e i file di configurazione.
sudo apt-get autoremove	Disinstalla package inutilizzati.
aptitude search *package*	Cerca il package nella lista locale.
wget *risorsa*	Scarica file dalla rete.
wget -r *risorsa*	Scarica ricorsivamente le risorse seguendo i link.
wget -l n *risorsa*	Indica di quanto addentrarsi al massimo ricorsivamente.
wget -p *risorsa*	Fa scaricare tutte le risorse per visualizzazione corretta di una pagina.
wget -k *risorsa*	Fa puntare i link alle pagine locali.
wget -np *risorsa*	Non fa risalire alle directory superiori di quella specificata.
sudo fdisk	Crea le partizioni stabilendo intervallo blocchi da usare, il tipo di filesystem.
sudo mkfs.*tipoFileSystem*	Crea il file system dentro una partizione.
lsmod	Elenca tutti i moduli attivi.
modinfo *nomeModulo*	Dice le informazioni sul modulo specificato.
sudo modprobe *nomeModulo*	Carica il modulo specificato.

2 Crittografia

Algoritmo pubblico/noto, robusto e chiavi segrete (o note in parte) di varie dimensioni.

- Codificare (E): $testoOriginale + chiaveCodifica \xrightarrow{codifica} testoCodificato.$
- Decodificare (D): $testoCodificato + chiaveDecodifica \xrightarrow{decodifica} testoOriginale.$

Impossibile ottenere testo orignale dal testo cifrato senza conoscere la chiave.

2.1 Crittografia Simmetrica (chiave segreta)

Stessa chiave K per codificare e decodificare per A e B.
+Veloce, +Efficente, -distribuzione chiavi fase risciosa, -tante chiavi (n*(n-1)/2).

$$A_K \xrightarrow{testo} E_K \xrightarrow{E_K(testo)} D_K \xrightarrow{testo} B_K$$

Algoritmo noto a tutti, chiave nota solo a A e B.

- SI: segretezza, autenticazione, integrità
- NO: Non repudiabilità

Adottato da Des, Triple-DES, AES, ecc.

2.2 Crittograffia Asimmetrica (chiave pubblica)

Una chiave pubblica/nota (K^+) e una chave privata/segreta (K^-) sia per A che per B **correlate**[1]. A e B non codividono chiavi segrete.

Due chiavi private-pubbliche (correlate) criptano e decriptano in modo reciproco. La chiave privata è incalcolabile dalla chiave pubblica.

+Sicurezza (non scambio completo chiavi), -Lento, -azzenza chiavi pubbliche proprietà.

Configurazioni:

- A sa a chi spedisce ma B non sa chi gli scrive.

$$A^{K_A^+}_{K_A^-} \xrightarrow{testo} E(K_B^+) \xrightarrow{K_B^+(testo)} D(K_B^-) \xrightarrow{testo} B^{K_B^+}_{K_B^-}$$

  SI: segretezza.

- A non sa chi capisce cosa scrive ma B sa chi scrive.

$$A^{K_A^+}_{K_A^-} \xrightarrow{testo} E(K_A^-) \xrightarrow{K_A^-(testo)} D(K_A^+) \xrightarrow{testo} B^{K_B^+}_{K_B^-}$$

  SI: Autenticazione, Integrità, Non repudiabilità.

- A sa chi capisce cosa scrive e B sa chi scrive.

$$\begin{tikzcd}
A^{K_A^+}_{K_A^-} \arrow[d, "testo"'] & B^{K_B^+}_{K_B^-} \\
E_1(K_A^-) \arrow[d, "K_A^-(testo)"'] & D_2(K_A^+) \arrow[u, "testo"'] \\
E_2(K_B^+) \arrow[r, "K_B^+(K_A^-(testo))"'] & D_1(K_B^-) \arrow[u, "K_A^-(testo)"']
\end{tikzcd}$$

  SI: Segretezza, Autenticazione, Integrità, Non repudiabilità.

Adottato da RSA, ecc.

2.2.1 Funzioni Hash Crittografiche

Una **Funzione Hash crittografiche ($MD()$)** (one-way hash, funzioni digest) prende in input un messaggio (m) lungo a piacere e da in output un digest (stringa) a lunghezza fissa ($MD(m)$).

Il digest non è reversibile, ossia non esise $MD(x) = MD(y)$ oppure dato $y = MD(x), x$ è indeterminabile.

Più due m sono simili più i digest sono diversi. Più due m sono diversi non è detto che i digest siano molto diversi.

Le hash non sono decodificabili, non servono chiavi di cifratura. Le funzioni di codifica/decodifica non restituiscono output a lunghezza fissa.

Hash è diverso per quanto simili dal checksum (perchè è reversibile).

Funzioni hash: MD5, SHA-1/2(x, 256, 512, ecc)/3, ecc.

2.3 MAC e Firma digitale

MAC (Message Authentication Code) è una tecnica, veloce, di autenticazione parziale dell'origine di un messaggio. Inoltre garantisce l'integrità del messaggio.

MAC: Si attua combinando crittografia simmetrica e funzioni Hash crittografico. Attori devono conoscere la chiave segreta. Assente autenticazione di chi spedisce (potrebbe essere l'altro o sestesso). SI: Integrità, autenticazione.

Firma Digitale: Tecnica di MAC avanzata. Si attua combinando crittografia asimmetrica e funzioni Hash crittografico. Mit. deve avere chiavi assimetriche. Chiunque può leggere e verificare l'integrità. SI: Integrità, Autenticazione, Non repudiabilità.

Spedizione:

[1] Un testo cifrato con una chiave può essere decrifrato solo dalla chiave a cui è correlata (legata).

1. Applico Funzione hash crittografico a m ottenendo MD(m).
2. Codifico il diget simmetricamente o asimmetricamente (con la chiave privata del mittente) ottenendo il MAC.
3. Concatenazione MAC al m.
4. Spedizione (m, MAC) che può modificare m in (m', MAC).

Ricezione:

1. Separazione di (m', MAC).
2. Decodifica simmetrica o asimmetrica (usa chiave pubblica mittente) del MAC ottenendo il digest (MD(m)).

   Terminazione improvvisa se decodifica non da successio. Implica che mittente non è quello atteso.
3. Applico Funzione hash crittografico a m' ottenendo MD(m').
4. Confronto i due digest. Se sono uguali m' è l'originale spedito dal mittente dichiarato.

$$
\begin{tikzcd}
m \arrow[r, "hash"] \arrow[rrr, bend left] & MD(m) \arrow[r, "codifica"] & MAC \arrow[r] & (m, MAC) \arrow[d, "Spedizione"'] \\
errori & successo & uguali? \arrow[l] \arrow[ul, bend left] & MD(m') \arrow[l] & m' \arrow[l, "hash"'] & (m', MAC) \arrow[l] \arrow[dl] \\
 & & & MD(m) \arrow[ul] & MAC \arrow[l, "decodifica"']
\end{tikzcd}
$$

2.4 GPG e PGP

GPG meccanismo di firma digitale gratuito basato su PGP proprietario. Garantisce autenticità e integrità di un file/package.

Il **maintainer**[2] della risorsa possiede chiavi asimmetriche e per ogni risorsa da pubblicare crea la firma digitale e la distribuisce con la risorsa. Oggi firmato solo file indice di risorse (Release.gpg) che ha versione e checksum (digest) di ogni risorsa.

L'utente, scarica se non l'ha la chiave pubblica, scarica il package+firma cerifica firma con la chiave pubblica del mainteiner.

Le chiavi pubbliche si possono trovare di default nell'OS o si scaricano da canali ufficiali verificando a modo.

[2] Qualcuno che distribuisce, assicura il funzionamento e aggiornamento di una risorsa online.

GPG	Descrizione
sha256sum *nomefile[, ...]*	Stampa una riga per ogni file pssato contenete il digest e il file separati da: '*' se file binario, ' ' se file di testo, 'U' se file universale, '?' per bits.
sha256sum -c *nomeFile.sha256*	Verifica correttezza del digest contenuto in nomeFile.sha256. Il rispettivo nomeFile deve essere nella direcotry corrente.
gpg –list-secret-keys	Verifica disponibilità chiavi GPG. Aggiungendo `--key-format long` si ha l'id delle chiavi.
gpg –gen-key	Genera le chiavi GPG. `gpg --full-generate-key` Maggiore configurazione.
gpg –armor –detach-sign nomefile	Crea file di firla nomefile.asc (firma detached).
gpg –detach-sign nomefile	Crea file binario nomefile.sig.
gpg –clearsign nomefile	Crea file nomefile.asc con testo e firma (firma inline).
gpg –sign nomefile	Crea file nomefile.gpg con testo e firma in binario (firma in-place).
gpg –verify nomefile.arc nomefile	Verifica firma detached.
gpg –verify nomefile.gpg|arc	Verifica firma inline o incorporate.
gpg –output nomefile.txt –decrypt nomefile.asc	Verifica e estae in nomefile.txt il nomefile.asc firmato inline.

Bisogna aver creato la cartella `.gnupg` nella home direcotry.

3 Programmazione Concorrente

Inclusioni necessarie:

- **#include <sys/types.h>** Definire tipi di dato speciali usati nelle chiamate di sistema.
- **#include <unistd.h>** Include funzioni e costanti del sistema operativo Linux/Unix.

Multi-processo	Descrizione
fork()	Crea un processo figlio. Condivide codice successivo alla fork e possiede una copia dei dati. Restituisce il pid del figlio al padre ($PID_F > 0$) e 0 al figlio ($PID_F = 0$) o un codice d'errore (¡0).
exit(0)	Termina un processo restituendo lo stato indicato come parametro (0 stato successo). Se padre termina prima del figlio non si possono liberare le risorse.
wait(NULL)	Fa attendere al processo la terminazione del processo figlio. Restituisce il pid del processo terminato.
wait(pidProcesso)	Fa attendere al processo corrente uno specifico processo.
waitpid(pidProcesso, &status, statoAttesso)	Fa aspettare un thread specifico con uno stato specifico.
exec	Sostituisce codice e dati a un processo. Non crea processi figlio.

Un processo possiede: Il proprio PID (gestito dall'os), il PID del processo figlio o 0 e il pid del processo "iziale" (gestito da descrittore di file).

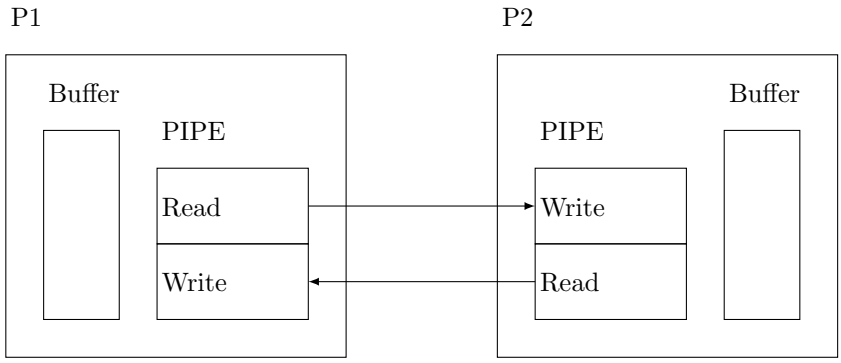
Struttura base programma multi-processo:

```
int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) { "gestione errore" }
    else if (pid == 0) { "processo figlio" }
    else { "processo padre" }
}
```

Ogni processo tiene referenza di un unico figlio, anche se ne può creare diversi. Quindi dopo ogni fork è buona cosa separare i flussi dei due processi.

PIPE	**Descrizione**
pipe(varPipe[2] : int) : int	Crea una pipe unidirezionale. Su varPipe[0] si leggerà. Su varPipe[1] si scriverà. Restituisce -1 in caso di errore, 0 se creazione avviene con successo.
close(varPipe[0|1])	Chiude una estremità della pipe.
read(varPipe[0], buffer, bufferSize)	Blocca esecuzione in attesa di dati da leggere.
strcpy(buffer, "...")	Prepara il buffer con il messaggio da inviare.
write(varPipe[1], buffer, sizeBuffer+1)	Scrive nella pipe.

Permette di comunicare fra processi correlati usando sistem-call e i descrittori di file. Il collegamento esiste fino a eliminazione esplicita o del processo.



Ogni processo ha un buffer di caratteri e una pipe (array di due celle) per scrivere e leggere con l'altro processo i dati contentuti nel buffer.

Nota: I processi sono visti come file, per questo le operazioni si chiamano come quelle dei file.

Multi-thread	**Descrizione**
nomeFunzioneThread(arg : void*) : void*	Funzione assegnata da eseguire a un thread. Necessita questa firma specifica per accettare e restituire qualsiasi tipo di dato. Serve eseguire un cast esplicito.
pthread_create(&varThread, &pthreadAttribut, tFunction, &args)	Crea un nuovo thread dentro al processo corrente. I parametri sono: 1. Variabile tipo thread. 2. Puntatore a struttura di attributi del thread. Default è NULL. 3. Funzione che sarà eseguita dal thread. 4. Puntatore a struttura contenente parametri usati dalla funzine.
pthread_join(&varThread, NULL)	Fa attendere processo la fine del thread indicato.

Struttura base programma multi-processo:

```
void *tFunction(void *args) {...}
int main() {
    pthread_t thread;
    ... args = ...;

    ptread_create(&thread, NULL, tFunction, &args);
}
```

Nota: La creazione di un processo è più lenta della creazione di un thread perchè nella prima bisogna creare un intero file descriptor, mentre nella seconda parziale.

3.1 SJF

Calcolo approssimato CPU Bust: $T_{n+1} = \alpha t_n + (1-\alpha)T_n$.

- t_m tempo n-esimo CPU burst. Storia recente.
- T_n previsione prevista. Storia passata.
- α peso storia recente e passata.

Calcolo Media esponensiale: $T_{n+1} = \sum_{j=0}^{n} \alpha(1-\alpha)^j t_{n-j} + (1-\alpha)^{n+1} T_0$.

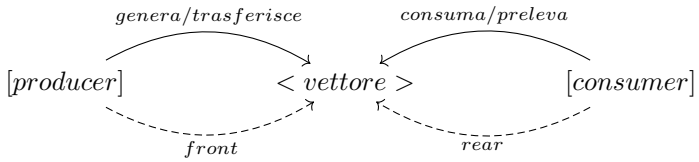
3.2 Produttore e consumatore

$$[producer] \xrightarrow{genera/trasferisce} < variable > \xleftarrow{consuma/preleva} [consumer]$$

```
shared Object buffer;

Semaphore empty =
   new Semaphore(1);

Semaphore full =
   new Semaphore(0);

cobegin

  Producer

//

  Consumer

coend
```

```
process Producer {
  while (true) {
    Object val = produce();
    empty.P();
    buffer = val;
    full.V();
  }
}

process Consumer {
  while (true) {
    full.P();
    Object val = buffer;
    empty.V();
    consume(val);
  }
```

3.3 Buffer limitato




```
Object buffer[SIZE];
int front = 0;
int rear = 0;
Semaphore empty =
  new Semaphore(SIZE);
Semaphore full =
  new Semaphore(0);
```

```
process Producer {
  while (true) {
    Object val = produce();
    empty.P();
    buf[front] = val;
    front = (front + 1) % SIZE;
    full.V();
  }
}
process Consumer {
  while (true) {
    full.P();
    Object val = buf[rear];
    rear = (rear + 1) % SIZE;
    empty.V();
    consume(val);
  }
}
```

3.4 Filosofi orientali a cena

$$pensa \overset{}{\leftrightarrow} [Filosofi] \xrightarrow{mangia} < chopsticks >$$

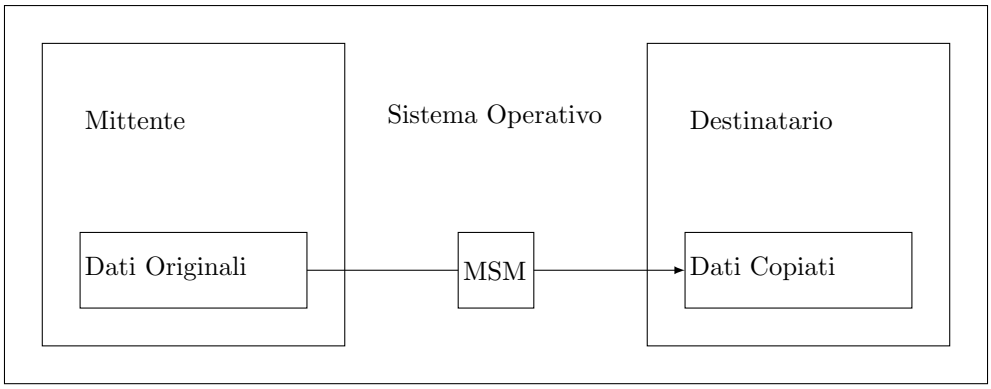
```
Semaphore chopsticks =
  { new Semaphore(1), ..., new Semaphore(1) };

process Philo[i] {  /* i = 0...4 */
  while (true) {
    think
    chopstick[i].P();
    chopstick[(i+1)%5].P();
    eat
    chopstick[i].V();
    chopstick[(i+1)%5].V();
  }
}
```

Le battecche sono un vetore lungo 5 (0...4) e i filosofi accedono alle battecche i e i+1 in %5.

Rischio di deadklock!!! Esistono varie soluzioni, avere un filosofo mancino è la più semplice:

```
process Philo[0] {
 while (true) {
  think
  chopstick[1].P();
  chopstick[0].P();
  eat
  chopstick[1].V();
  chopstick[0].V();
 }
}
```

3.5 Message Passing



Legenda: p=mittente, m=messaggio, q=destinatario.

Sincrono:

- Mittente attende l'ACK (Acknowledgement) del destinatario.
  `ssend(m,q)`
- Destinatario attende la spedizione di m da p o da un qualsiasi mittente (*). Poi manda l'ACK.
  `m=sreceive(p|*)`

Asincrono:

- Mittente non attende l'ACK del destinatario.
  `asend(m,q)`
- Destinatario attende la spedizione di m da p o da un qualsiasi mittente (*). Non manda l'ACK.
  `m = areceive(p|*)`

Completamente asincrono:

- Mittente non attende l'ACK del destinatario.
  `asend(m,q)`
- Destinatario non attende la spedizione di m da p o da un qualsiasi mittente (*). Non manda l'ACK.
  `m = nb-receive(p|*)`
  Se non si riceve nulla restitusce `null`.

Nella pratica: lo scambio di messaggi avviene tra canali di comunazioni su cui mandare e ricevere. L'istruzione `replay(Thoth)` sblocca il mittente (e non la receive).

3.6 Concorrenza in C

Usata la librerira POSIX **semaphore.h** (API POSIX con semafori, lock mutex e variabili condizionali).

Tipi di semaforo:

- Con nome (**named**) utile a sincronizzare processi non correlati.
- Senza nome (**unnamed**).

Funzioni semafori	**Descrizione**
sem_t *< *variabile* >	Definisce una variabile di tipo puntatore a struttura Semaforo.
sem_ini(&< *varSem* >, 0/altro, ¡valoreIniziale¿)	Crea un semaforo unnamed inizializzato al valaore passato e lo aseegna alla variabile d'indirizzo indicata. Secondo parametro a 0 semaforo condiviso tra thread stesso processo, a $\neq$ 0 conviso tra processi.
sem_open("nomeSem", 0_CREAT,0666, ¡valoreIniziale¿)	Crea e restituisce un semaforo named in R/W (3° parametro) inizializzato al valore passato.
sem_wait(&varSem)	Esegue l'operazione P (decrementa).
sem_post(&varSem)	Esegue l'operazione V (incrementa).
sem_destroy(&varSem)	Distrugge un semaforo.

Le CS saranno contenute tra un wait e un post.

4 Esercizi Scheduling

Procedura risolutiva di un esercizio di scheduling data la **durata del quanto di tempo** e una tabella dei processi del tipo:

Processi	Tempo di Arrivo	Tempo d'Esecuzione
...	...	...

Passo 1
Fare una tabella di scheduling come la seguente:

Ready Queue Running Quanto d'Inizio/Fine				
Tempo	0	1	...	n
T.R. P1 ... T.R. PN				

Spiegazione delle righe della tabella:

1. *Ready Queue* specifica per ogni quando i processi pronti per esere eseguiti, tipicamente è FIFO.
2. *Running* indica per ogni quanto di tempo il processo in esecuzione.
3. *Quanto d'Inizio/Fine* evidenzia i quanti d'arrivo (A) e di termine (F) dei processi.
4. *Tempo* sono i quanti del processore. Ogni colonna è un quanto.
5. *Righe T.R.* tempo d'esecuzione rimante per ogni processo a ogni quanto di tempo dalla sua partenza.

Passo 2
Per rispondere alle statistiche richieste fare una tabella come la seguente:

Processi	T. Turnaround	Intervalli di Ready	T. Attesa
...	...	[x1-x2], ...	...
Medie:	...	...	...

Spiegazione delle colonne della tabella:

1. *Turnaround* = T. fine - T. arrivo.
2. *Intervalli di Ready* sono i quanti in cui il processo è stato nella coda di ready (dal quanto in cui vi è entrato al quanto in cui vi è uscito).
3. *T. Attesa* somma degli intervalli di ready.

5 File Descriptors

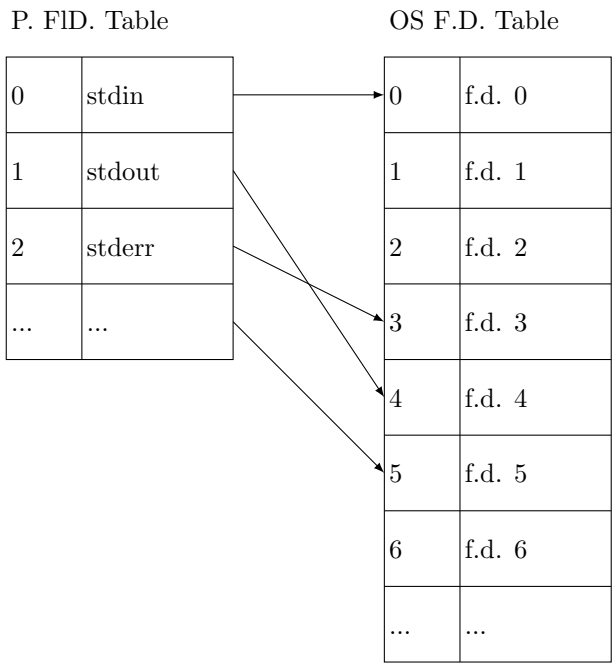
Il file descriptor è un'astrazione (integer, inizio buffer) dell'os, e quindi multi linguaggio, per permettere l'accesso ai file.

Ogni processo ha la **file descriptor table** di record indiretti delle informazioni (file descriptor) sui file aperti dal processo stesso. I record sono indiretti perchè puntano ai record della **tabella di sistema**. La tabella di sistema contiene file descriptor dei file attualmente aperti.

Un processo/shell figlia eredità una copia della f.d. table, successivamente modificabile. Padre e figlio si devono sincronizzate sull'accesso ai file. Ciò permette un modo di comunicazione tra i processi.

Rindizionamenti:

- **Processi figlio**: Padre può cambiare le associazioni fd/stream passate al figlio.
- **Auto-ridirezionamento**: fd viene associato a stream diverso.



In POSIX per ogni processo i file descriptor standard sono: stdin=0, stdout=1, stderr=2.

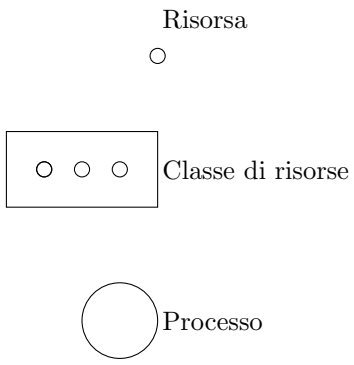
Ricoda che stdin è lo stream input da tastiera e stdout e stderr sono gli stream a video dell'output e dei messaggi d'errore.

Nota: Processi diversi possono avere file descriptor diversi, ma constesso valore, e che referenziano file diversi.

In C per scrivere si usa ¡fprintf(stdout/stderr, "..." [, parametri])¿ e per leggere si usa ¡fgets(buffer, size, stdin)¿, per il fine file usare ¡feof(stdin)¿.

6 Risorse

6.1 Grafo di Holt Generale (classi e processi)

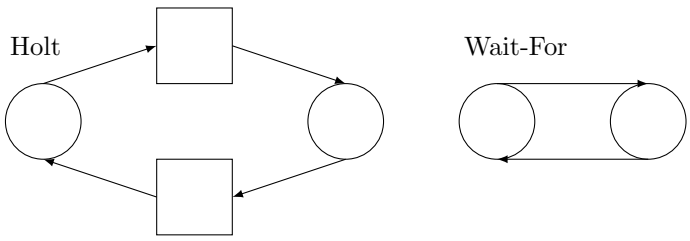


Per gli archi si può segnare la molteplicità, e nelle classi il numero di risorse non assegnate.

6.2 Detaction and recoverty - Caaso 1

Teorema: Siano le risorse ad'accesso mutualmente esclusivo, seriali e non prerilasciabili. Con una risorsa per classe si ha deadlock $\iff$ Holt contiene un ciclo (attesa circolare).

Dimostrazione: In un grafo di Holt si ha attesa circolare se si ha attesa circolare nella rispettiva variante di grafo Wait-For (elimina le risorse collassando gl'archi appropriati).



6.3 Detaction and recoverty - Caaso 2

Teorema: Siano le risorse ad'accesso mutualmente esclusivo, seriali e non prerilasciabili. Con più risorse per classe si ha deadlock $\iff$ Holt non è completamente riducibile (eliminazione tutti archi).

6.4 Detaction and recoverty - Knot

Teorema: Siano le risorse ad'accesso mutualmente esclusivo, seriali e non prerilasciabili. Con una richiesta sospespa per processo si ha deadlock $\iff$ esiste un knot.

6.5 Stato SAFE - algoritmo del banchiere

Sia s una permutazione dei valori $1, \ldots, N$ e $s(i)$ l'i-esima posizione nella sequenza. Si calcola il vettore **avail** cosi:

1. avail[1]=SC
2. avail[1+j]=avail[j]+$p_{s(j)}$, con i=$1, \ldots, N-1$

Uno stato del sistema è safe se vale: $n_{s(j)}$¡=avail[j], con j=1,. . .,N.

Con il banchiere a singola valuta basta ordinare in modo crescente gli n_i per assicurare lo stato SAFE.

Nel caso multi valuta si aggiunge il pedice k della valuta.

Con il banchiere multi valuta ogni vlauta ha un suo ordine.

7 Gestione Pacchetti in Linux

7.1 Concetti chiave

- **Package (.deb)**: archivio compresso di file descrivono programma e metadati del package.
- **Dipendenze**: Librerie necessarie funzionamento programma.
- **Repository**: Collezioni locali o remote di package in server centralizzati.
- **Database locale di package**: Elenco locale dei metadati di package nei repository.

7.2 Installazione Package

Interfaccia a CLI per download-installazione (risolve dipendenze e gestisce fonti), rimozione, aggiornamento, gestione di package. Usa `dpkg` per l'installazione vera e propria (disassembla package e installa in directory giuste) e `apt-cache` per la ricerca (usabili anche dall'utente).

Assente di descrizione esecuzioni (ideale per script).

`apt-get`

`apt` fornisce una descrione testuale della procedura eseguita (user freandly).

Comandi apt-get	Descrizione
update	Aggiorna database locale di package. Da eseguire per primo.
upgade	Installa le nuove versioni dei package già installati.
install	Installa un package nuovo e le dipendenze.
remove	Rimuove package lasciando file configurazione.
purge	Rimuove package e file configurazione.
autoremove	Rimuove package, non più necessari, autoinstallati soddisfare dipendenze altri package.
dist-upgrade	Aggiorna la distribuzione (package e dipendenze).
clean	Cancella package dalla cache locale.

Formato generico: `sudo apt-get "comando" "packageOpzionale"`

Procedura Installazione Package:

1. `sudo apt-get update` Aggiorna l'indice/database locale scaricando l'elenco aggiornato dei package e versioni dai repository.
2. `apt-get intall "package"` Installa il package cercandolo nel database locale e controllando le dipendenze.

7.2.1 Posizione Repository

I database locali di package sono contenuti in: `/etc/apt/sources.list` (principale), `/etc/apt/sources.list.d/` (aggiunte repository individuali).

Formato one-line: `deb [opzioni] URI distribuzione comp1 comp2 ...`

- **Tipo**: Indica il tipo d'archivio.

  `deb` indica package binari precompilati.

  `deb-src` indica package di sorgenti.

- **Opzioni**: La firma GPG si attua col campo dedicato che accettà il **keyring** con la public-key del mainteiner.

  `signed-by="..."`

- **URI**: Indirizzo del repository.

- **Distribuzione**: Nome in codice della versione del OS.

- **Componenti**: Definiscono il tipo di licenza (main, restricted, universe, multiverse).

Caratteristiche: chiarezza, leggibilità, facile da gestire e sicurezza esplicità.
C'è anche un formato deb822 più leggibile.

Directory public-key: `/usr/share/keyrings/` (key di sistema), `/etc/apt/keyrings/` (key d'utente).

Il `keyrings, sources.list, sources.list.d/` sono editabili con editor con privilegio sudo o GUI specifiche.
Aggiunta Manuale:

1. Verificare esisetnza directory, scarica GPG-key (sopprime output e reindirizza allo stdout), comprimerla (ASCII in binario .gpg) e salvataggio nel keyring.

   ```
   sudo mkdir -p /etc/apt/keyrings
   wget -qO- URI | sudo gpg --dearmor -o directory.gpg
   ```

2. Aggiungere riferimento one-line con signed-by del repository alla directory.

   ```
   echo "FormatoOneLine" | sudo tee directory.list
   ```

3. Aggiornamento e installazione: `sudo apt update`, `sudo apt install nomePackage`.